

Recurrencias — Preguntas y respuestas del curso

Preguntas surgidas a lo largo del curso, organizadas por tema. Cada pregunta nació de forma natural al estudiar el material — ese es el mejor indicador de qué conceptos merecen profundización.

1. Herramientas (Tools)

¿Existen más herramientas además de las tres del curso (`get_current_datetime`, `add_duration_to_datetime`, `set_reminder`)?

Esas tres son **herramientas de ejemplo inventadas para el curso**, no herramientas reales de Anthropic. El sistema de tools es un mecanismo genérico — vos podés crear cualquier herramienta que necesites. Hay dos categorías:

- **Herramientas custom:** las definís vos con Python y un schema JSON. Las tres del curso son ejemplos de esto.
- **Herramientas built-in de Anthropic:** vienen integradas en la API y Anthropic se encarga de ejecutarlas:

Herramienta	Qué hace
<code>web_search</code>	Búsqueda en tiempo real en internet
<code>text_editor</code>	Leer y editar archivos de texto
<code>computer_use</code>	Controlar mouse, teclado y pantalla
<code>bash</code>	Ejecutar comandos de terminal
Ejecución de código	Correr Python en un contenedor Docker aislado

¿De dónde saca la información `get_current_datetime`? ¿De la misma PC?

Sí. La función llama a `datetime.now()` de Python, que lee el reloj del sistema operativo de la máquina donde corre el código:

```
from datetime import datetime

def ejecutar_herramienta(nombre, params):
    if nombre == "get_current_datetime":
        return datetime.now().isoformat() # ← reloj de la PC/servidor
```

Claude **nunca toca el reloj directamente** — él solo pide que alguien se lo informe. Tu código es quien accede al sistema operativo y le devuelve el dato. Si el agente corre en un servidor en otro país, devuelve la hora de ese servidor, no la del usuario.

El JSON dentro de Python, ¿se introduce con corchetes?

Depende de qué parte del JSON estés mirando — en Python hay dos estructuras:

Estructura JSON	En Python	Símbolo
Objeto {}	dict	Llaves {}
Array []	list	Corchetes []

```
# La lista de herramientas es una list → corchetes
tools = [herramienta_a, herramienta_b]

# Cada herramienta individual es un dict → llaves
herramienta_a = {
    "name": "get_current_datetime",
    "description": "Obtiene la fecha actual",
    "input_schema": {          # dict anidado → llaves
        "type": "object",
        "properties": {},
        "required": []         # lista vacía → corchetes
    }
}
```

Regla práctica: pares clave-valor → {}. Elementos en secuencia → [].

2. Web Search y Scraping

La herramienta de búsqueda con `allowed_domains`, ¿es una forma de scraping?

No exactamente. Son mecanismos distintos:

Aspecto	Web search (built-in)	Web scraping
Qué hace	Consulta un motor de búsqueda y recibe snippets	Descarga el HTML de una página y extrae datos
Quién ejecuta	Anthropic a través de su API	Tu código con requests HTTP
<code>allowed_domains</code>	Filtra en qué sitios puede buscar Claude	No aplica
Requiere permiso del sitio	No	Depende del <code>robots.txt</code> y los ToS

`allowed_domains` es simplemente un filtro que le dice al motor de búsqueda que solo devuelva resultados de ciertos dominios — útil para que Claude solo cite fuentes confiables.

Para scraping real existe `firecrawl-mcp-server` — un servidor MCP que descarga el contenido completo de páginas y se lo pasa a Claude como contexto. Eso sí es scraping; la web search built-in no lo es.

¿El servidor de Firecrawl requiere autorización de la página que scrapea?

No. Lo que necesitas es:

Qué necesitas	De quién
API key de Firecrawl	firecrawl.dev (te registrás ahí)
Nada	Del sitio que scrapeás

Firecrawl accede a los sitios como cualquier navegador. Sin embargo, que no requiera autorización no significa que siempre sea legal — muchos sitios prohíben el scraping en sus términos de servicio. Firecrawl respeta `robots.txt` por defecto.

Para fuentes que sí requieren autenticación (Sheets, bases de datos, APIs privadas), existen otros servidores MCP específicos:

Fuente	Servidor MCP	Autenticación
Google Sheets / Drive	<code>mcp-google-drive</code>	OAuth de Google
Bases de datos SQL	<code>mcp-postgres</code> / <code>mcp-sqlite</code>	Credenciales de la DB
Sitios públicos	<code>firecrawl-mcp-server</code>	Solo API key de Firecrawl

Si solo quiero leer datos de un Sheet ajeno para copiarlos a mi base de datos, ¿necesito permiso de edición?

No — solo necesitas permiso de **lectura (Viewer)**:

Permiso	¿Necesitás este?
Viewer (lector)	☒ Suficiente para leer y copiar
Editor	Solo si querés escribir de vuelta en su Sheet
Owner	No necesario

El permiso Viewer también es más fácil de conseguir — la empresa está más cómoda otorgándolo porque no hay riesgo de que tu sistema modifique sus datos.

3. Files API

¿La Files API es como un atajo para no volver a cargar los archivos y que estén disponibles para ser usados?

Exacto. Sin la Files API subís el mismo archivo en cada request. Con ella lo subís una vez y solo referenciás el ID:

```
# Una sola vez
file_id = client.beta.files.upload("reporte.pdf")

# Todas las veces siguientes – solo el ID, sin re-subir el archivo
{"type": "document", "source": {"type": "file", "file_id": file_id}}
```

La combinación más poderosa es **Files API + Prompt Caching**: el archivo no se re-sube y el procesamiento inicial no se re-cobra si el contexto está en caché.

¿La Files API es similar a cuando en Claude subís un archivo a un Proyecto?

Sí — es el mismo concepto en dos interfaces distintas:

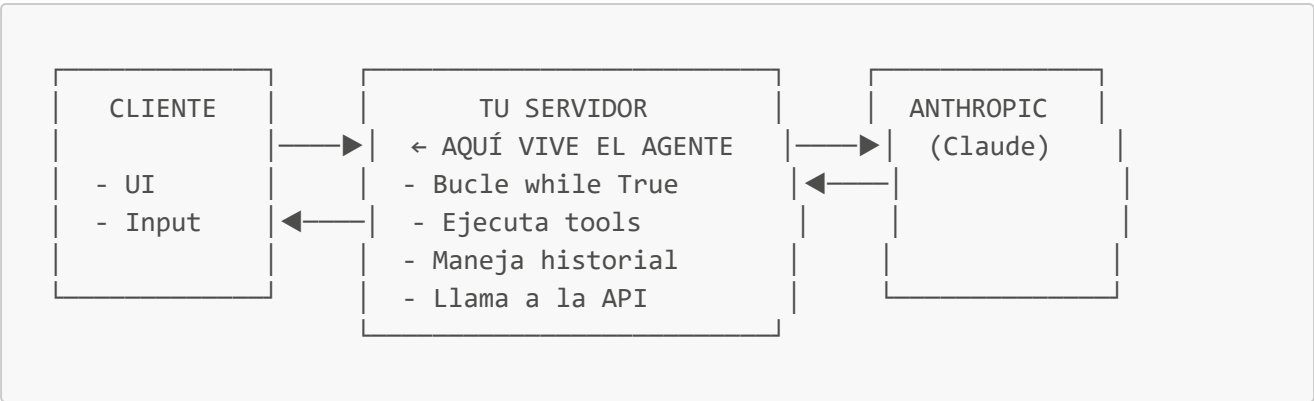
	Proyectos de Claude (UI)	Files API (código)
Quién lo usa	Usuario en claude.ai	Desarrollador vía API
Cómo se sube	Drag & drop	<code>client.beta.files.upload()</code>
Persiste entre sesiones	Sí	Sí
Control de cuándo usarlo	No — Claude siempre lo ve	Sí — vos decidís en qué requests incluirlo

Cuando subís un PDF a un Proyecto en claude.ai, Anthropic internamente hace algo muy similar a la Files API. La Files API simplemente te expone ese mismo mecanismo para usarlo en tu propio código.

4. Arquitectura: cliente, servidor y agentes

¿El agente está dentro del servidor? ¿Los servidores contienen agentes?

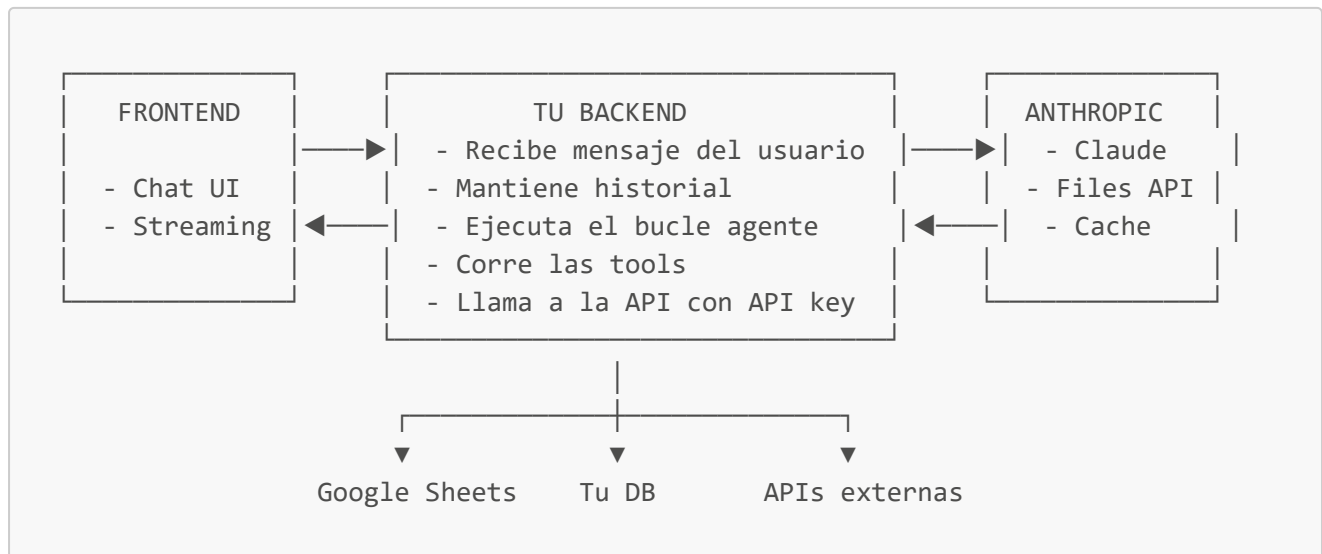
El agente vive en **tu servidor (backend)**, no en los servidores de Anthropic ni en el cliente:



Anthropic provee el **modelo** (Claude) y la **API**. El agente es el código que vos escribís alrededor: el bucle, las herramientas, la lógica de orquestación. Claude es el cerebro, el sistema nervioso es tuyo.

¿Puedo crear un frontend con un chatbot, un backend con agentes y tools, y usar Claude via API key?

Sí — esa es exactamente la arquitectura estándar de una aplicación de IA en producción:



Vos construís y sos dueño del frontend, el backend, la lógica de negocio y los datos. Le comprás a Anthropic el poder computacional del modelo — solo pagás por los tokens que usás.

¿En el ecosistema de Claude se puede reemplazar claude.ai como frontend y tener servidores propios con un plan Enterprise?

Sí. Hay tres niveles de integración:

NIVEL 1 – Usar Claude.ai tal cual

└ Frontend, backend y modelo: todo de Anthropic

NIVEL 2 – Tu frontend, backend de Anthropic (lo que el curso enseñó)

└ Frontend y backend: tuyos. Modelo: Claude via API key

NIVEL 3 – Enterprise / VPC deployment

└ Frontend y backend: tuyos. Claude deployado en tu propia infraestructura

El plan Enterprise permite deployar Claude en tu propia nube (AWS, GCP, Azure) — los datos no salen de tu infraestructura. La variable que lo habilita es:

```
ANTHROPIC_BASE_URL=https://tu-empresa.internal/claude
```

Tu backend llama a tu propio endpoint en vez de a los servidores de Anthropic. Para el código, el cambio es transparente.

¿Con un modelo open source como Ollama necesito mi propio frontend?

Sí — y además tenés todo el stack, no solo el frontend:

	Claude (Anthropic)	Ollama / Open source
Modelo	De Anthropic, vos lo usás	Tuyo — corre en tu hardware
Frontend	Claude.ai o el tuyo	Solo el tuyo
API key	Obligatoria	No necesitás
Costo por token	Sí	No — solo el costo del hardware
Privacidad	Los datos viajan a Anthropic	Todo queda en tu máquina

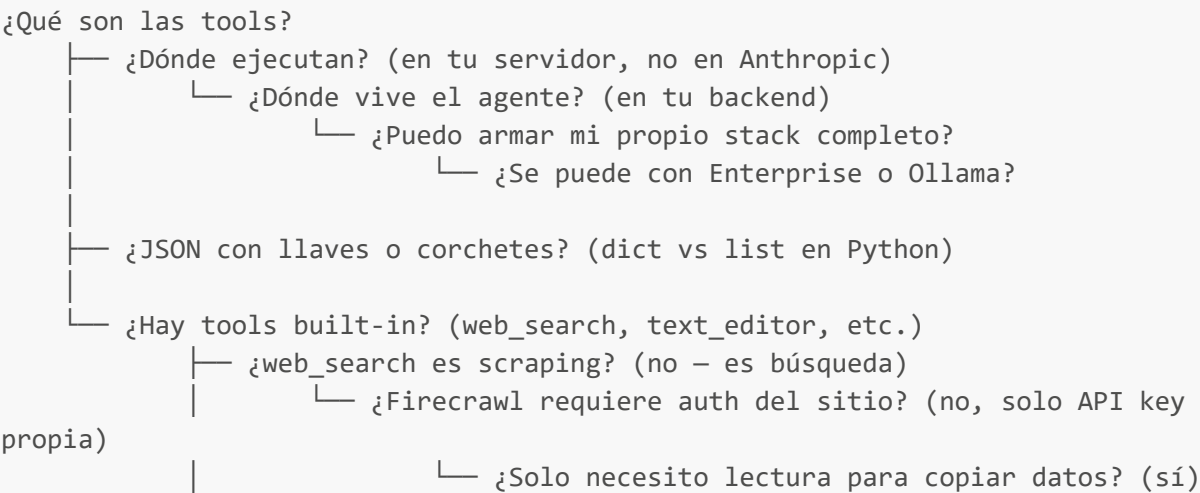
La buena noticia es que la lógica de agentes, tools e historial que aprendiste en el curso es transferible — solo cambia el cliente:

```
# Con Claude
import anthropic
client = anthropic.Anthropic(api_key="sk-...")

# Con Ollama (compatible con la misma interfaz)
from openai import OpenAI
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
```

Cuándo usar Claude	Cuándo usar Ollama
Máxima capacidad de razonamiento	Datos que no pueden salir de tu empresa
Prototipo rápido sin infraestructura	Costo cero por token, alto volumen
Integración con Files API y cache	Regulaciones estrictas de privacidad (salud, legal)

Mapa de relaciones entre preguntas



- └ Files API (subir una vez, reusar siempre)
 - └ ¿Es como los Proyectos de Claude? (sí, misma idea)